YAML

```
name: webdev-deployment
description: Choose and configure a Manus website's deployment architecture, st
```

Website deployment and performance

Use this skill before choosing a Web project's production serving arrangement, and when changing
that arrangement or investigating a delivery problem. Work with the existing framework and the
user's requirements. The main Webdev skill owns development, validation and publication authorization; this skill supplies deployment choices and contracts.

Choose how content is produced and served

For good user-perceived performance and experience, usually prefer static delivery of frontend
build output with backend APIs for dynamic data when that fits the application. Treat this as a starting point, not a fixed architecture. SEO and discoverability, first-page content, personalization, freshness, framework constraints or an explicit user preference for SSR can justify a different choice. Choose the rendering and serving approach that best meets those requirements, and explain the relevant trade-off in the plan.

Application needs	Usual starting point	Published request handling
Page content can be produced before a visit	Static HTML or static generation (SSG)	Serve the generated pages and assets from static storage.
Interactive UI obtains its business data from APIs	Browser-rendered frontend (often an SPA/CSR) plus an API backend	Publish frontend files as static output and send dynamic requests to the application server.
Initial HTML must be generated for the current request	Server rendering (SSR) for those pages	Send those page requests to the server; serve independently publishable assets from static storage.

Preserve explicit user choices and existing framework requirements; an existing application does not need an unrelated migration. For SEO, both static generation and SSR can provide

complete HTML. Choose between them according to the content and delivery requirements, using
webdev-seo when search discoverability is part of the work.

SPA describes browser navigation; CSR, SSR and SSG describe where or when page content is
generated. R2 is the platform's object storage for built files, not a renderer or an API server. An Express server returning an empty HTML shell and JS is still a browser-rendered application.

Static frontend files and dynamic APIs can share one public origin through path routing. Application login, a database and dynamic API data do not by themselves require SSR or frontend
files to be served by the same process. The API still authenticates requests and enforces access
to private data. Use the server path when the HTML itself must be generated for that request;
do not confuse a static application shell with the private data it later obtains from an API.

Record the selected serving arrangement and cache policies in the implementation plan. Identify
the actual build output and dynamic paths; naming a framework alone does not make these choices.

Build and publication setup

Serving arrangement	Required declarations
Static output only	`build`
Application container only	`features.server: true` and `deploy`
Static frontend plus container backend	`build`, `features.server: true`, and `deploy`; declare `routes` for the intended path split

For a mixed project, publish the static output as well as the container. A frontend build inside
a Dockerfile only puts files into that image; it does not upload them to static storage. Use distinct frontend/backend build targets when that avoids unnecessary work, while reusing
the project's existing build system and committed dependency policy.

- `build` declares a self-contained `command` and a repository-relative `outputDirectory` that contains `index.html`. The platform runs the command on a clean checkpoint, with no implicit
  dependency installation, then uploads that output. See the static build contract.
- `deploy` declares the Dockerfile location and an application health path. The image owns its
  compilation, production entrypoint and listener. See the container contract
  for defaults, platform environment delivery, resource lifetime and failure handling.
- Config transport, revision checks and publication authorization remain in configuration and checkpoints.
  Resource sizes and idle residency are in hosting.

Declaration requirements

These requirements also apply when this Skill is supplied before a configuration write. Each domain write replaces that whole domain, using the current configuration revision.

Field	Accepted declaration
`build.command`	Non-empty after trimming, at most 1000 characters; no inline credential literals. The publication pipeline also checks bytes and rejects NUL. Include dependency installation when needed.
`build.outputDirectory`	Repository-relative subdirectory, 1–256 characters from `A-Za-z0-9._/-`; no absolute path, `.` / `..` segments, empty segments or trailing slash. It must contain `index.html`; symlinks must remain inside the repository.
`deploy.dockerfilePath`	Optional, defaults to root `Dockerfile`; same canonical relative-path restrictions.
`deploy.healthPath`	Required application path starting with `/`, 1–256 characters from `A-Za-z0-9._/-`, without `..` or NUL; its handler must return an unauthenticated 2xx–3xx.
`buildCache`	Optional top-level container image-cache setting, enabled by default; requires `deploy`. It does not control browser caching or static builds.

Store `features.server: true` before a separate `deploy` write. Container startup comes from the Dockerfile `ENTRYPOINT` / `CMD`; do not declare `deploy.port`. Use the existing backend's Dockerfile/PORT contract. Static publication runs `sh -eu -c "<build.command>"` at the checkpoint
root and has a documented five-minute budget for clone, build and upload together. Detailed
build environment, runtime and failure contracts remain in the linked reference.

Route according to that choice

Match the project's real paths in first-match order. Dynamic APIs, health handlers, webhooks
and server-rendered pages must reach the server before any static catch-all. Asset paths go to
their published output. SPA fallback is only for browser-managed page routes; a missing asset
or API endpoint must not be disguised as an HTML success response.

For an SPA with an API, use this default routing pattern with the project's actual paths:

JSON

```
{"routes":[{"path":"/api/*","target":"server"},{"path":"/assets/*","target":"st
```

Add the application's other dynamic paths before the catch-all. For SSR, send the relevant page
paths to `server` instead of applying an SPA catch-all to them. Same-origin requests remain same-origin even when the gateway selects different serving paths.

`routes` changes take effect on the next publish, not on the development Preview. Static targets
require `build`; server targets require `features.server`. SPA fallback is static-only. Server rules may declare short shared HTML caching. The complete path syntax, reserved paths, validation rules and observed fallback
behavior are in routing and responses.

The ordered table contains 1–32 rules. Each rule has a `path` and `target` (`static` or `server`). Paths start with `/`, use `A-Z a-z 0-9 . _ ~ -` in segments, have at most 256 characters, and may end in `/*`; `/*` is the catch-all. Bare `/`, dot-only or empty segments, a trailing slash, mid-path wildcards, query/fragment delimiters and whitespace are invalid. Duplicate paths and
rules fully covered by an earlier wildcard are rejected. A domain fragment `{}` removes the table; an empty `routes` array is invalid.

Static rules may carry `cache: "immutable"`, `"no-cache"`, or integer seconds from 0 to 31536000,
and a boolean `spaFallback`. Literal rules at or below `/favicon.ico`, `/manus-oauth/`, `/api/scheduled/`, `/manus-storage/`, `/__manus/`, `/__system__/`, `/sitemap.xml` and `/robots.txt` are reserved. Broader wildcards can cover them, but platform handling has priority. The schema
has no User-Agent or arbitrary-header matching condition.

Cache public server-rendered pages

For public HTML that is the same across visitors and tolerates a short delay in content updates,
keep `target: "server"` and use the existing `cache` seconds declaration:

JSON

```
{"routes":[{"path":"/blog/*","target":"server","cache":300},{"path":"/*","targe
```

Use `webdev.config` with `GET config`, preserve unrelated rules, then `PUT config/routes` with the complete `{"routes":[...]}` body. This takes effect after publication, not in Preview.
Server cache accepts integer seconds **0–600**, or `"no-cache"`; 0 and `"no-cache"` disable it. `"immutable"` remains static-only. Omitting `cache` preserves existing behavior.

For a public page intended to use this shared cache, the application can omit `Cache-Control` or send `public, max-age=0`. Do not add `private`, `no-store`, or `no-cache` to that response; they prevent platform storage even when the route declares a positive cache duration. Keep those protections on personalized or sensitive responses. Browser-facing HTML still receives the platform's `no-store` policy.

Only public production GET requests returning HTTP 200 HTML outside `/api/` can enter this shared cache. Requests with cookies or authorization bypass it; the application's `private`, `no-store`, `no-cache`, `Set-Cookie`, or unsupported `Vary` also prevent storage. Never declare shared caching for personalized content. The platform caches by deployment and keeps browser
responses uncacheable. After publishing, repeat an anonymous request and check `X-Manus-Cache`
(`MISS` then `HIT`); `BYPASS` means the request or response was ineligible. The complete conditions and verification are in server caching.

Choose cache policy by content

Response content	Default cache policy

Public assets whose URLs contain a content hash or version	Long-lived caching, commonly `public, max-age=31536000, immutable`. Change the URL when the bytes change.
Public HTML at a stable page URL	For a server route with shared caching, follow the application-header requirements above. Otherwise use `no-cache` with a validator such as ETag, or the platform's uncached HTML handling.
Mutable resources with unchanged URLs	Revalidate or use a short freshness lifetime that fits the content.
Personalized responses that may be stored in the user's browser	`private, no-cache` when freshness requires validation before reuse.
Sensitive responses that must not be stored	`private, no-store`.
Shared public API data	Cache only when its allowed staleness and response variations are understood; choose that explicit lifetime.

Under HTTP semantics, `no-cache` allows storage but requires validation before reuse. The platform's server HTML cache instead treats an application's `no-cache` as a storage bypass;
it does not implement conditional revalidation. `no-store` forbids storage.
`private` prevents shared-cache reuse. Authentication cookies alone do not establish the policy.

Choose caching by response content, not by whether the application uses SSR. Public SSR output
that is reusable across users can be cached when its freshness requirements permit; personalized
output must not enter a shared cache. SSR page requests do not change the long-lived caching
default for versioned public JS, CSS and images.

Apply policies to the appropriate paths or content types. A long-lived asset policy must not also cover mutable HTML merely because both files occupy the same output directory. Conversely,
an API's `no-store` policy must not blanket versioned frontend assets. Prefer the framework's existing header and conditional-request support.

The table gives application defaults, not a promise that the gateway emits those exact headers.

The platform transforms some responses, and a stored `routes.cache` value alone is not serving
evidence. Platform defaults, HTML injection and CSP compatibility are documented under platform HTTP behavior.

Keep the first load focused

Load the current page's required code first. Defer substantial optional screens and capabilities
until they are used, with the existing framework's lazy loading and code splitting. Splitting source into files does not reduce startup downloads if the entrypoint still imports everything.
Choose asset sizes appropriate to their display and avoid serial requests that have no data dependency; preserve authentication and actual dependency order.

Verify the affected delivery behavior

Use source inspection and existing build output first. For a deployment or performance change
that requires serving evidence, inspect representative page, asset and API responses after the
authorized publish: destination, content type, cache headers, direct links and missing assets.
Keep unchanged-version repeat visits distinct from first loads and from visits after a publish.

A successful config write or aggregate publish does not prove static output was uploaded or a
rule was applied. Use the serving checks for that boundary.
When measuring speed, keep conditions comparable and separate document wait, asset transfer and
data requests. Report what was measured; no configuration choice guarantees a particular load time.
Do not turn this into an extra build or browser-testing pass for every routine edit.